



PRODUCT SPECIFICATION & QA REFERENCE

CBC Thane Worship Portal

A complete reference to how the application is built, how each feature behaves, and how to verify it — written for engineers picking up the codebase and testers signing off a release.

VERSION

1.0

STATUS

Live & in active development

PREPARED FOR

Engineering & QA

DATE

25 July 2026

cbc-worship-portal.pages.dev · Confidential — internal engineering document

Document Control

Document title	CBC Thane Worship Portal — Product Specification & QA Reference
Version	1.0
Prepared	25 July 2026
Audience	Developers extending or maintaining the codebase; testers validating releases
Source material	Direct review of the live codebase (src/, apps-script/Code.gs), the project's own architecture reference, and the current engineering task brief
Live deployment	https://cbc-worship-portal.pages.dev
Repository scope	React 18 + Vite 5 frontend, Google Apps Script backend, Google Sheets database

How to read this document. Sections 1–4 describe what the product is and how it is organised — start here if you are new to the codebase. Sections 5–8 are the technical reference (data, API, automation, integrations) — use these while coding. Section 9 covers non-functional behaviour. Section 10 is the QA checklist — use it directly when validating a build. Section 11 lists things that are known to be incomplete or risky. Where this document describes work that is *planned but not yet built*, it is explicitly marked **PLANNED** so it is never confused with current production behaviour.

A note on accuracy. This document was produced by reading the application's source files directly — page components, the Apps Script backend, and its own internal architecture notes — rather than from a general description of the product. Where the codebase and its own documentation disagreed (for example, the change log references EmailJS while the backend in fact uses GmailApp for all transactional mail), this document follows the code, and the discrepancy is flagged in Section 11.

Contents

01	Product Overview	4
02	System Architecture	6
03	Users, Roles & Hierarchy	8
04	Feature Modules	11
05	Data Model	20
06	Backend API Reference	23
07	Notifications, Email & Automation	27
08	External Integrations	29
09	Non-Functional Requirements	31
10	Test Plan & QA Checklist	33
11	Known Gaps & Technical Debt	37
12	Deployment & Release Process	39
13	Appendix	41



SECTION 01

Product Overview

What the app is, who it serves, and the single design principle every change must respect.

CBC Thane Worship Portal is a production progressive web app built for the worship team of CBC Thane — a 48-plus member volunteer team — to run the day-to-day operations of the ministry: notices, prayer, equipment, attendance, the weekly prayer roster, birthdays, talent recruitment, and member onboarding. It replaced a mix of WhatsApp threads and spreadsheets with a single installable app that works on phones, tablets, and desktops.

14

FRONTEND PAGES

17

GOOGLE SHEETS
TABLES

60+

BACKEND ACTIONS

5

SCHEDULED
AUTOMATIONS

What it does

The app gives every team member a single home for: reading and RSVPing to notices, posting and praying over prayer requests, checking equipment in and out and logging maintenance, viewing event attendance, seeing the weekly prayer-facilitator roster and Zoom details, celebrating birthdays, suggesting new talent for the team, and managing their own profile. Admins additionally create events, mark bulk attendance, manage the equipment inventory, post notices, assign the roster, and (as Super Admin) configure the whole system.

Who it is for

AUDIENCE	WHAT THEY NEED FROM THE APP
Regular members	A fast, low-friction way to stay informed and participate — notices, prayer, attendance visibility, birthdays — without ever feeling like they are using an “admin tool.”
Department admins	The ability to action items in their area (equipment, events, roster, auditions) without extra permissions elsewhere.
Super Admin	Full technical control — Firebase configuration, backups, Vision & Mission content, quiz questions, and system-wide settings.

The governing design principle

DEAD SIMPLE FOR MEMBERS

The team is explicitly optimising for 95% adoption across the volunteer base. Every enhancement under active development is required to be *additive on the admin side only* — a regular member’s day-to-day flow (view notices, RSVP, acknowledge, pray, submit a request or suggestion) must look and feel exactly as it does today. No new permission prompts, no new navigation items, and no added friction are to be introduced for the base member experience. This principle is stated explicitly in the project’s own engineering brief and should be treated as a hard constraint by anyone extending the app.

Technology at a glance

LAYER	CHOICE
Frontend	React 18, Vite 5, Tailwind CSS, React Router 6
Backend	Google Apps Script (single <code>Code.gs</code> file, deployed as a Web App)
Database	Google Sheets (17 sheets act as tables, read/written via SpreadsheetApp)
Hosting	Cloudflare Pages (static build output)
Photo storage	Cloudflare R2 (profile, asset, and maintenance photos)
Push notifications	Firebase Cloud Messaging, v1 API
Transactional email	Gmail, via Apps Script's GmailApp
Install experience	Installable PWA with a service worker, offline app-shell caching, and custom Android/iOS install prompts

WHY APPS SCRIPT AND SHEETS

This architecture trades raw scalability for zero infrastructure cost and a database (a spreadsheet) that non-technical team leaders can open and read directly. It is well suited to a single-team, sub-100-user deployment, but it carries real constraints — no transactions, limited concurrency safety, and an API surface defined entirely by a large `switch` statement. These constraints are called out throughout this document wherever they affect behaviour a tester should know about.



SECTION 02

System Architecture

The request path from a tap in the app to a row written in a spreadsheet, and everything in between.

Every read and write in the app goes through one HTTP endpoint: the Apps Script Web App deployment URL. There is no separate API server, no ORM, and no application database beyond the spreadsheet itself.

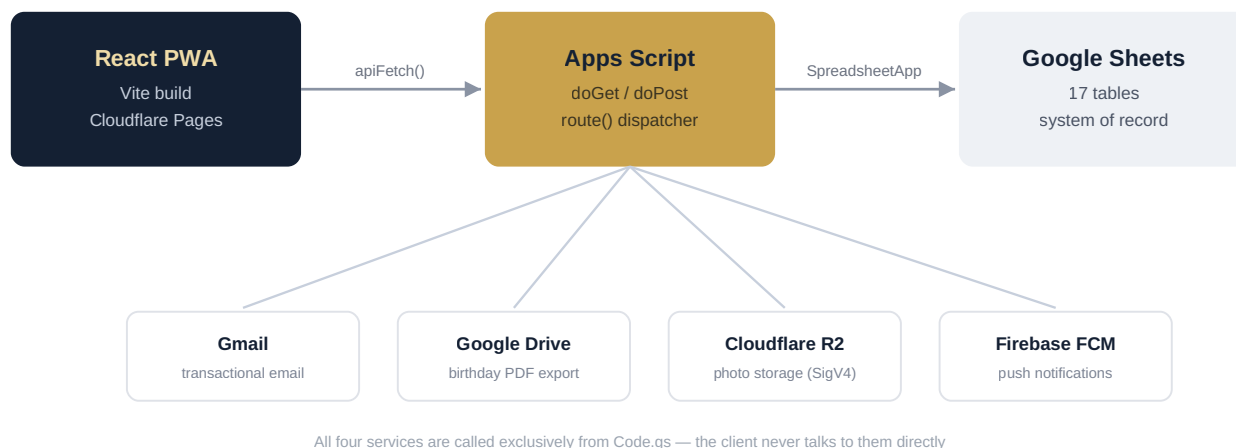


Figure 2.1 — Request path and external service boundary. The client only ever calls the single Apps Script Web App endpoint.

Request lifecycle

1. A page component calls `apiFetch(action, payload)` from `src/utils/api.js`.
2. `apiFetch` injects the current session token from `localStorage`, checks a `sessionStorage` cache for cacheable read actions, and serialises the payload.
3. Requests that include a `photoBase64` field are sent as a JSON `POST`; every other request is sent form-encoded. The backend's `doPost` tries a JSON parse first and falls back to `e.parameter`.
4. `route(action, p)` in `Code.gs` dispatches to one of 60+ named handler functions via a single `switch` statement.
5. Handlers call `requireAuth(p, role)` where applicable, read/write the relevant sheet(s) via shared helpers (`sheetToObjects`, `appendRow`, `updateRow`, `findRowById`), and return `ok(data)` or `err(message)` as JSON.
6. On success, mutating handlers call `touchCache(sheetName)`, which writes a `cacheTs_<Sheet>` timestamp into the Settings sheet.

Client-side caching

Cacheable `GET`-style actions are cached in `sessionStorage` keyed by action name and parameters. After any mutation, the frontend calls `invalidateCache(action)` to clear the relevant cache entries before reloading. The Dashboard additionally keeps its own 60-second time-boxed cache (`dash_cache_<memberId>`) so repeat visits within a minute render instantly without a network round trip; pull-to-refresh on mobile forces a fresh fetch.

Code splitting

All authenticated pages except `LoginPage` and `SetupWizard` are loaded via `React.lazy`, so the initial bundle only contains the shell, auth, and routing logic. Each page ships as its own chunk (visible as separate files under `dist/assets/`), which keeps first paint fast on the mobile connections most members are using.

First-run setup

On a fresh deployment with no members yet, the app renders `SetupWizard` instead of the login screen. The wizard has two steps: paste and test the Apps Script Web App URL (calls the unauthenticated `ping` action), then create the team name and the first Super Admin account (`initialSetup`, which also runs `ensureSheets()` to create all 17 required sheets with correct headers if they do not already exist).



SECTION 03

Users, Roles & Hierarchy

The role model as it runs in production today, the permission matrix behind it, and the department-scoped model that is planned but not yet built.

Understanding roles correctly is the single most important thing for testing this app well: almost every page renders different content, and every backend handler enforces different rules, depending on who is asking. This section documents the role model exactly as implemented today, then separately documents the department-scoped model that is planned but has not yet been built, so the two are never confused during testing.

Governing principle

EVERYONE IS A MEMBER FIRST

There is no isolation between roles. Every single person — including every admin and the Super Admin — gets the full base member experience: notices, prayer board, event and attendance visibility, birthdays, and the ability to raise a request or suggestion on anything in the app. Admin status only ever *adds* the ability to action things; it never removes anything a member already had.

Current role model (live in production)

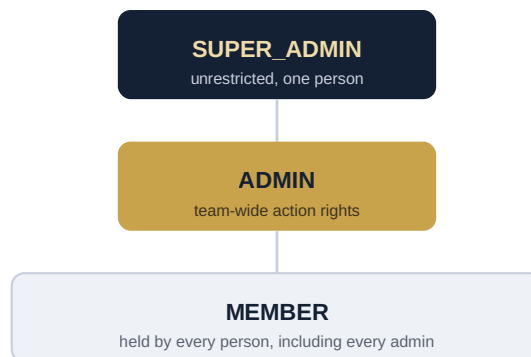


Figure 3.1 — Role model as implemented today: a flat, three-level enum on each member record.

The **Members** sheet stores a single flat **role** column with three possible values. There is no department scoping today — an **ADMIN** can action equipment, events, notices, roster, and auditions alike.

ROLE	WHO HOLDS IT	WHAT IT ADDS OVER MEMBER
MEMBER	Every person in the system	Baseline: view/participate in notices, prayer, assets, events, roster, birthdays, auditions; edit own profile
ADMIN	Team leads / worship leaders	Post & manage notices; manage equipment and maintenance; create events and mark bulk attendance; assign the roster; review and respond to talent suggestions; export attendance CSV is Super-Admin only

ROLE	WHO HOLDS IT	WHAT IT ADDS OVER MEMBER
SUPER ADMIN	Joey (developer/owner) — singular, rare	Everything ADMIN has, plus: Settings page (Vision & Mission content, quiz questions, onboarding checklist, Zoom/WhatsApp links, Firebase config), data export/import, clear-all-data, CSV export, manual birthday-PDF trigger

Permission matrix

Reflects the effective UI and backend behaviour observed in `Layout.jsx` (nav/FAB role filters) and `Code.gs` (`requireAuth` calls) as of this document's date.

AREA	MEMBER	ADMIN	SUPER_ADMIN
View notices / prayer / assets / events / roster / birthdays / auditions	Yes	Yes	Yes
Post notice, edit/delete any notice	—	Yes	Yes
Add / edit asset, log maintenance, mark maintenance complete	Log only	All	All
Create event, mark bulk attendance	—	Yes	Yes
Assign weekly roster slot, edit Zoom details	—	Yes	Yes
Review & update talent-suggestion status	—	Yes	Yes
Export attendance CSV	—	—	Yes
Settings page (V&M, quiz, onboarding checklist, Firebase config)	—	—	Yes
Data export / import / clear-all-data	—	—	Yes
Sidebar / FAB "Settings" entry point visible	No	Yes	Yes

Session & authentication mechanics

- Passwords are hashed client-side with SHA-256 before ever leaving the browser (`src/utils/sha256.js`); the server only ever sees and stores the hash.
- `authenticateUser` matches email + password hash against an active `Members` row, mints a new `sessionToken` (UUID v4), and sets `sessionExpiry` to now + 7 days — independent of the "Stay signed in" checkbox on the login screen, which only controls whether the token is persisted to `localStorage` or kept in memory.
- Every authenticated action calls `requireAuth(p, requiredRole)` server-side, which validates the token, confirms the account is active and the session has not expired, and enforces the role hierarchy (SUPER_ADMIN > ADMIN > MEMBER).
- Password reset is a two-step token flow: `requestPasswordReset` emails a reset link/token; `confirmPasswordReset` validates the token and writes the new hash. The reset form also enforces an 8-character minimum client-side.

KNOWN GAP — NOT EVERY READ ACTION REQUIRES A SESSION

A number of read actions have no `requireAuth()` call at all and will return data to *any* caller with the Apps Script URL, session or not: `getAssets`, `getAsset`, `getAnnouncements`, `getPrayerRequests`, `getAnsweredPrayers`, `getYearEndSummary`, `getEvents`, and `getMemberAttendance`. This is documented in the codebase's own architecture notes and is a live gap, not a planned change — see Section 11 for the full list and testing implications.

Planned role model — department-scoped admins PLANNED

The current engineering brief for this codebase specifies a richer role model that has **not been implemented yet** as of this document’s date. It is included here so that developers and testers share a single reference for where the system is headed, and so a tester never mistakes today’s flat role enum for a bug against the planned model.

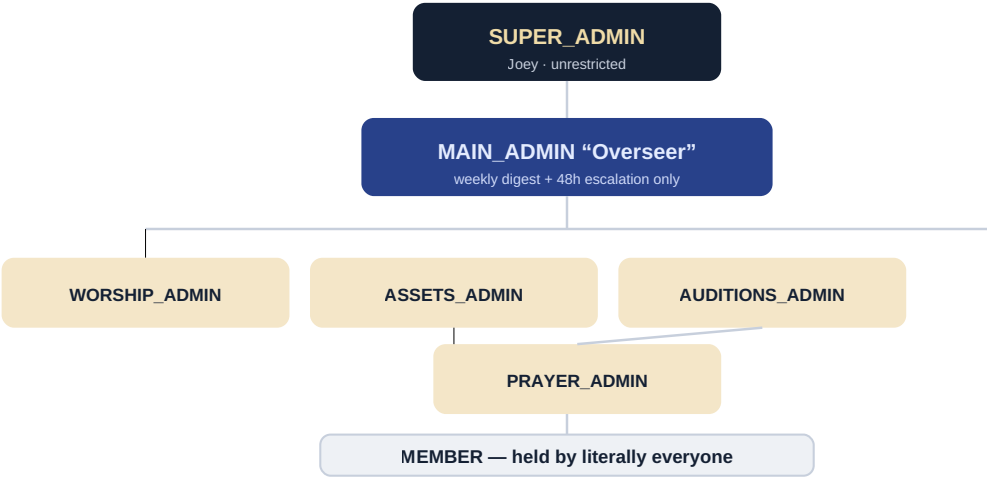


Figure 3.2 — Planned department-scoped role model (from the current engineering brief, not yet implemented). A person can hold zero or more department grants alongside MEMBER.

PLANNED ROLE	SCOPE
SUPER_ADMIN	Unchanged — unrestricted, full technical access
MAIN_ADMIN “Overseer”	One tier below Super Admin, not tied to a single department. Does not receive routine per-item notifications; instead gets one weekly cross-department digest plus real-time escalation when an item sits unactioned past 48 hours
WORSHIP_ADMIN / ASSETS_ADMIN / AUDITIONS_ADMIN / PRAYER_ADMIN	Each a normal member plus action rights scoped to their own department only — e.g. the Assets admin can action maintenance requests but has no special rights over notices, roster, or auditions unless separately granted
MEMBER	Base role, held by literally everyone including every admin — unchanged from today

Per the brief, the `Members` sheet is to be extended to support a person holding `MEMBER` plus zero or more department-admin grants, rather than a forced single-role enum, since one person could plausibly administer more than one department. Existing `ADMIN` members are to be migrated to the closest department(s) based on actual usage, with ambiguous cases flagged for manual review. The brief explicitly notes that the current sheet schema has no obvious “Worship” action queue the way Assets, Auditions, and Prayer already do (via `MaintenanceLog`, `AuditionSuggestions`, and `PrayerRequests/FacilitatorRoster` respectively) — what belongs to a Worship queue is an open question to be resolved before that piece is built.



SECTION 04

Feature Modules

Every page in the app: purpose, user flow, business rules, and the edge cases worth testing.

Fourteen pages make up the member-facing and admin-facing surface of the app. Each is documented below with its access level, the primary user flow, the business rules that govern its behaviour, and — where they exist — the edge cases most worth probing in testing.

1 Login, Setup & Password Reset

2 Onboarding & Vision/Mission Sign-off

3 Dashboard

4 Notice Board

5 Prayer Board

6 Assets & Maintenance

7 Events & Attendance

8 Prayer Roster

9 Birthday Wall

10 Talent / Audition Suggestions

11 Profile

12 Notifications Centre

13 Settings (Admin)

14 Year-End Summary

1. Login, Setup & Password Reset

src/pages/LoginPage.jsx, SetupWizard.jsx

NO AUTH

Email/password sign-in, forgot-password flow, and the one-time first-run wizard that provisions the team.

User flow

1. Member enters email + password; "Stay signed in for 7 days" is checked by default
2. Password is SHA-256 hashed client-side, sent to `authenticateUser`
3. On success, session token is stored and an FCM push-permission request is fired in the background
4. Forgot password: enter email → reset link emailed → open link (token in URL) → set new password

Business rules

- Login is blocked (generic error) on wrong email/password/inactive account — no account enumeration
- New password must be at least 8 characters and match its confirmation field, enforced client-side
- First-run only: if no members exist yet, the app shows the two-step Setup Wizard (connect Apps Script URL, then create team + first Super Admin) instead of the login screen

Edge cases & things to test

- Reset token expiry / already-used token should show a friendly error, not a crash
- Test the wizard's `ping` connectivity check with an invalid Apps Script URL
- Confirm session actually expires at 7 days server-side, not just client-side

2. Onboarding & Vision/Mission Sign-off

src/pages/OnboardingFlow.jsx, VMReaderPage.jsx

MEMBER

A mandatory multi-step wizard every new member must complete before reaching the main app; also reused monthly for the ongoing V&M review streak.

User flow

1. Step 1 of 3: read Vision, Mission, and Expectations & Values content
2. Step 2: type the exact confirmation key phrase (configured in Settings) to confirm commitment
3. Step 3: work through an admin-defined checklist; required items must all be checked
4. On completing all required items, `completeOnboarding` fires automatically and the member is routed into the app

Business rules

- `RequireOnboarded` in App.jsx redirects any MEMBER with `isOnboarded=false` to `/onboarding` before they can reach any other route
- Typed confirmation is matched case-insensitively, trimmed
- Completing step 2 immediately records a V&M review for the current month, independent of finishing the checklist
- Rich text V&M content is sanitised client-side (script tags and inline event handlers stripped) before being rendered with `dangerouslySetInnerHTML`

Edge cases & things to test

- A member who is admin-created but never onboarded should be unable to see any nav item except the onboarding flow
- Confetti/progress bar should reflect only required items, but checklist completion tracks all items
- VMReaderPage is the same content reader used post-onboarding for the recurring monthly review, gated by a one-time sign-off per period

3. Dashboard

src/pages/Dashboard.jsx

MEMBER

Role-aware landing page. Every member sees the same personal cards; ADMIN and SUPER_ADMIN see additional operational summaries.

User flow

1. Loads from a 60-second sessionStorage cache first, then refreshes from `getDashboard` in the background
2. Pull-to-refresh (swipe down at scroll-top on mobile) forces an uncached reload
3. Member cards: greeting, V&M status badge, urgent-notice banner, prayer partner card, latest non-urgent notice
4. Admin-only: asset/member stat cards, maintenance alerts, V&M sign-off tracker, upcoming roster, pending audition count
5. Super-Admin-only: shortcuts to Settings and Year-End, last backup date

Business rules

- Birthdays, badges, and the personal attendance bar are implemented but explicitly disabled in the current build (dead code guarded by `{false && ...}`) — do not expect them on the live dashboard even though the components exist
- Urgent notices are shown in a red banner above everything else; non-urgent notices show as a normal card
- V&M badge reads “due” until the member’s monthly review is recorded

Edge cases & things to test

- Confirm the 60s cache is per-member (keyed by member id) so switching accounts on the same device doesn't leak stale data
- Confirm admin-only cards never render for a MEMBER even if the API response happened to include the fields

4. Notice Board

src/pages/NoticeBoardPage.jsx

ADMIN to post · MEMBER to respond

Team-wide announcements with per-urgency response requirements and read/acknowledge/RSVP tracking.

User flow

1. Admin posts a notice with a title, body, and one of four urgency levels: Info, Important, Urgent, Meeting
2. Opening the page marks all current notices as read for that member
3. Info and Urgent notices show an “Acknowledge” button
4. Meeting notices show Yes/No RSVP buttons instead of Acknowledge
5. Any member can share a notice to WhatsApp via a pre-filled message link
6. Admin can see collapsible lists of who has seen / acknowledged / RSVP'd yes / RSVP'd no, by name

Business rules

- RSVPing “No” to a Meeting notice requires a typed reason of at least 10 characters before the confirm button enables
- A notice is visually flagged “New” if created today or yesterday (IST)
- Urgent notices get a red left-accent border and an inline warning icon
- Important notices show neither Acknowledge nor RSVP controls — they are informational only
- Deep-linking to `/notices?id=<id>` scrolls to and highlights that notice for 1.5 seconds

Edge cases & things to test

- Acknowledge/RSVP use optimistic UI updates that roll back on API failure — test with a forced network failure to confirm the rollback and error toast both fire
- Only ADMIN and SUPER_ADMIN can post, edit, or delete; delete requires a confirmation dialog

5. Prayer Board

src/pages/PrayerBoardPage.jsx

MEMBER

A living board of active prayer requests plus a separate archive of answered ones with testimonies.

User flow

1. Post a request with a title, optional detail, and an anonymous toggle
2. Any member can tap “Pray for this” once per request; the tap is debounced and disabled after use
3. The request owner or any admin can mark a request answered, which requires a testimony of at least 20 characters
4. Marking answered moves the request from the Believing tab into the Answered tab and archives it to `AnsweredPrayers`

Business rules

- Anonymous requests hide the poster’s name behind an “Anonymous” label everywhere, including to admins in this view
- The pray button is disabled once the current member’s id appears in the comma-separated `prayingMembers` list, and is additionally rate-limited to one tap per second client-side
- Believing tab badge shows a live count of active (non-answered) requests
- Only the requester or an admin sees the delete / mark-answered controls on a card

Edge cases & things to test

- Testimony character counter must visibly gate the submit button below 20 characters
- Double-tapping “Pray” rapidly should never double-count — verify both the 1-second debounce and the in-flight lock

6. Assets & Maintenance

src/pages/AssetsPage.jsx

ADMIN to add/edit · MEMBER to log maintenance

Equipment inventory with category-specific maintenance checklists, photo capture, and full maintenance history per item.

User flow

1. Admin adds an asset (name, category, condition, purchase info, optional photo); the system auto-generates a human-readable Asset ID
2. Any member can log maintenance against an asset: date, maintenance type (filtered by the asset’s category), description, who did it, cost, next-due date, optional photo
3. Photos are compressed client-side, then uploaded in a second request after the record is created
4. Admin marks a maintenance log complete from the asset’s detail drawer
5. Search, category/condition filters, and date-range filters (last maintenance, next due) narrow the list

Business rules

- Asset ID = first 3 letters of the category (uppercase) + a zero-padded sequence number within that category

- Maintenance type options are category-specific (e.g. Guitars offers “String replacement”, “Fret polish”; Microphones offers “Capsule clean”, “Wireless transmitter battery check”); unmatched categories fall back to a generic list
- “Done by” accepts any active member or a free-typed “Other...” name
- Assets with a `nextDueDate` in the past show an “Overdue” badge; within 7 days shows “Due this week”; within 30 shows “Due this month”
- Logging maintenance with a next-due date also updates the parent asset's own `nextDueDate` field

Edge cases & things to test

- If the photo upload step fails after the asset/log record already saved, the UI must show a warning toast (not lose the record) — verify this recovery path specifically
- Photo capture must reject/compress oversized images to under 200KB at a maximum 800px dimension before upload
- Only ADMIN can mark a log complete; a MEMBER should see a read-only “Open” badge instead of the action button

7. Events & Attendance

`src/pages/EventsPage.jsx`

ADMIN to create/mark · **MEMBER** to view

Event creation and a tap-to-cycle bulk attendance marker that emails every purely-absent member automatically.

User flow

1. Admin creates an event: type (Sunday Worship, Rehearsal, etc.), date, optional notes
2. Admin opens “Mark” on an event and taps each member's row to cycle Present → Informed Prior → Absent → Present
3. Saving calls `markAttendanceBulk`, which writes a row for *every active member*, not just the ones tapped
4. Absent members (status = Absent, i.e. neither Present nor Informed Prior) automatically receive a “We missed you” email
5. Super Admin only: export all attendance history as CSV

Business rules

- Default status for any member not yet tapped is Absent
- “Informed Prior” members are treated as excused and do *not* receive the absence email, but are still recorded as not-present
- Attendance rows are upserted per event: re-marking an already-marked event updates existing rows rather than duplicating them

Edge cases & things to test

- Because the save rewrites all active members' rows for that event, adding a new member *after* an event was already marked will not retroactively create a row for them until the event is re-marked
- Verify the absence email is sent only to purely-Absent members and never to Informed-Prior or Present members

- Confirm the on-screen Present/Informed/Absent running counts stay in sync while tapping quickly

8. Prayer Roster

src/pages/RosterPage.jsx

ADMIN to assign · **MEMBER** to view

An 8-week rolling view of who is facilitating the Monday 9:30pm prayer meeting, plus the shared Zoom details.

User flow

1. Page always shows the next 8 upcoming Mondays, starting this week
2. Admin taps the edit icon on any week to assign or clear a facilitator from a member dropdown
3. Admin edits the shared Zoom link, meeting ID, and passcode from the same page
4. “This week” and “You!” badges highlight the current week and the signed-in member’s own slot

Business rules

- A week can be flagged “On Break” via a **Settings** value listing break months — this overrides any assignment display for that week
- Unassigned weeks show as italic “Unassigned” rather than blank

Edge cases & things to test

- Roster week-matching normalises both stored Date objects and ISO strings to a plain **YYYY-MM-DD** before comparing — a good place to check for timezone drift bugs around midnight IST

9. Birthday Wall

src/pages/BirthdayPage.jsx

MEMBER

A two-tab celebration wall: today’s birthdays (with a blessing wall) and the next 30 days of upcoming birthdays.

User flow

1. Today tab lists every active member whose birthday matches today’s date (DD/MM, IST) with a text box to leave a blessing
2. Once a member posts a blessing for a given person, the box is replaced with a confirmation and hidden going forward
3. Upcoming tab lists everyone with a birthday in the next 30 days, sorted soonest-first, with a countdown

Business rules

- A birthday member does not need to post to themselves — the wall shows for others to write on
- Minimum wish length is 2 characters (effectively just “not empty”)
- Today’s-birthday matching and days-until math are both computed against IST, not the browser’s local timezone

Edge cases & things to test

- Verify a member cannot post two blessings to the same person on the same birthday (UI hides the form after posting, but this should also hold if reloaded mid-session)

- A daily scheduled trigger (see Section 7) separately emails each birthday member a PDF of their wall's blessings that evening — test this independently of the in-app wall

10. Talent / Audition Suggestions

src/pages/AuditionsPage.jsx

MEMBER to suggest · ADMIN to review

A lightweight recruitment pipeline where any member can suggest a person for the team and admins track follow-up status.

User flow

1. Member submits a suggestion: name, skill/instrument, ministry (Worship/DLP/Sound/Stage/Other), description, contact info
2. Non-admins only ever see suggestions they personally submitted; admins see and can search all of them
3. Admin opens a suggestion and sets its status (Following Up, Invited to Audition, On Hold, Not Suitable) plus an optional response note

Business rules

- Status colour-coding: Following Up = info blue, Invited to Audition = amber, On Hold = neutral, Not Suitable = red
- The submitter list filter matches on the submitter's stored *name*, not member id

Edge cases & things to test

- Confirm a MEMBER genuinely cannot see other members' suggestions, not just that the UI hides the search box

11. Profile

src/pages/ProfilePage.jsx

MEMBER

Self-service profile editing, password change, badge display, attendance percentage, and PWA install controls.

User flow

1. Member edits their own details and uploads a profile photo (compressed client-side, then uploaded to Cloudflare R2)
2. Change-password form re-hashes both current and new password client-side before sending
3. Displays the member's own attendance percentage and any earned badges
4. Surfaces the browser push-permission request and the PWA install prompt / iOS install instructions

Business rules

- A MEMBER can only ever update their own record — the backend explicitly rejects `updateMember` calls where the target id does not match the caller's id, unless the caller is ADMIN or above

Edge cases & things to test

- Re-uploading a photo should overwrite the previous R2 object at the same key rather than accumulating orphaned files

12. Notifications Centre

src/pages/NotificationsPage.jsx

MEMBER

In-app notification inbox surfaced via the bell icon on both the mobile bottom nav and desktop header.

User flow

1. Shows the last 30 days of notifications for the signed-in member
2. Mark a single notification read, or mark all read
3. Unread count badge polls in the background and displays on the mobile nav's centre "Alerts" tab and the bell icon

Business rules

- Every notification links somewhere (`linkTo`) — tapping should route directly to the relevant item, e.g. a maintenance log or a specific notice

13. Settings (Admin)

src/pages/SettingsPage.jsx

SUPER ADMIN

The single admin console for system-wide configuration: members, content, and integrations.

User flow

1. Manage members: add, edit, deactivate
2. Edit Vision & Mission content, onboarding quiz questions, and the onboarding checklist
3. Configure Zoom link/ID/passcode, WhatsApp link, Firebase config, and the onboarding confirmation key phrase
4. Export or import the full dataset, or clear all data

Business rules

- Every action on this page requires `SUPER_ADMIN` — it is the only page not reachable by `ADMIN`
- Clear-all-data and import are destructive and should always be behind an explicit typed confirmation

Edge cases & things to test

- Data export/import round-trip should be tested against a copy of the sheet, never the live production spreadsheet

14. Year-End Summary

src/pages/YearEndPage.jsx

NO AUTH

read · triggered by

SUPER ADMIN

A shareable, filterable retrospective of every prayer answered in a given year.

User flow

1. Filter answered prayers by year
2. View is designed to be shared outside the app (e.g. read aloud or exported) as a testimony retrospective

Business rules

- Reads only from **AnsweredPrayers**, so a request must have been explicitly marked answered (with its testimony) to appear here



SECTION 05

Data Model

All 17 Google Sheets that make up the database, grouped by what they're for.

Google Sheets is the system of record — there is no separate database. Each of the 17 sheets below is one tab in the spreadsheet, with the first row as headers and every row from row 2 down read into an object via `sheetToObjects()`. There are no foreign-key constraints or transactions; referential integrity (e.g. a `memberId` that still exists) is entirely the responsibility of the Apps Script handlers.

TESTING IMPLICATION

Because writes are simple row appends/updates with no locking beyond what Apps Script provides automatically, concurrent edits from two admins at once (e.g. two people marking the same event's attendance simultaneously) are a realistic scenario worth testing explicitly — the last write wins per row.

Core

Identity and system configuration.

SHEET	PURPOSE	KEY FIELDS
Members	Every user account: identity, credentials, role, onboarding state, session, and push token.	id, name, email, passwordHash, phone, gender, instrument, birthday (DD/MM), bio, photoUrl, role, joinDate, isOnboarded, onboardingStep, fcmToken, lastVMReview, vmReviewStreak, sessionToken, sessionExpiry, isActive
Settings	System-wide key–value configuration store.	teamName, confirmKeyPhrase, quizQuestions (JSON), onboardingChecklist (JSON), firebaseConfig (JSON), zoomLink, zoomMeetingId, zoomPasscode, whatsappLink, birthdayTime, rosterReminderDays, Drive folder ids, lastBackupDate, vmVision / vmMission / vmValues, cacheTs_<Sheet> timestamps

Content

Member-generated content that appears directly in the UI.

SHEET	PURPOSE	KEY FIELDS
Announcements	Notice board posts and their per-member response state.	id, title, body, urgency, createdBy, createdAt, readBy (names), acknowledged (names), rsvpYes / rsvpNo (names), rsvpReason (JSON name → reason map)
PrayerRequests	Active (Believing-status) prayer requests.	id, title, detail, isAnonymous, postedBy, postedAt, status, prayingMembers (comma-separated ids), testimony, answeredAt
AnsweredPrayers	Archive of answered requests, source for the Year-End Summary.	id, originalRequestId, title, detail, postedBy, answeredAt, testimony

SHEET	PURPOSE	KEY FIELDS
BirthdayWishes	Individual blessing-wall messages.	id, birthdayMemberId, year, wisherId, wisherName, wish, createdAt

Operations

The operational queues behind each department (equipment, events, roster, recruitment).

SHEET	PURPOSE	KEY FIELDS
Assets	Equipment inventory.	id, assetId (human-readable), name, category, subcategory, description, serialNumber, condition, assignedTo, purchaseDate, estimatedValue, status, photoUrl, notes, checkedOutBy/At/Purpose, nextDueDate, lastMaintDate, updatedBy
MaintenanceLog	Every maintenance record raised against an asset.	id, assetId, serialNumber, date, maintenanceType, description, doneBy, cost, nextDueDate, isCompleted, completedAt/By, raisedAt, submittedBy, photoUrl
Events	Worship services, rehearsals, and special events.	id, type, date, notes, createdBy
Attendance	Per-event, per-member attendance status.	id, eventId, memberId, isPresent, status (Present / InformedPrior / Absent), markedAt
FacilitatorRoster	Weekly prayer-meeting facilitator schedule.	id, memberId, weekDate, memberName, notes, reminderSent, assignedBy, notificationSent, reminderNotificationSent
AuditionSuggestions	Talent recruitment pipeline.	id, suggestedName, skill, ministry, description, contact, submittedBy/ById, submittedAt, status, adminResponse, respondedAt
PrayerPartners	Seasonal member-to-member prayer pairings.	id, member1Id, member2Id, season, pairedAt, isActive

System

Supporting records not directly authored by members.

SHEET	PURPOSE	KEY FIELDS
Notifications	In-app notification records.	id, memberId, type, title, body, isRead, createdAt, linkTo
Badges	Member achievement badges.	id, memberId, badgeKey, badgeName, badgeEmoji, awardedAt, isCustom
OnboardingProgress	Per-member onboarding-checklist completion state.	id, memberId, step, isCompleted, completedAt

SHEET	PURPOSE	KEY FIELDS
VMReviews	Vision & Mission sign-off log (initial + monthly).	id, memberId, completedAt, score, month, year



SECTION 06

Backend API Reference

Every one of the 60-plus Apps Script actions, grouped by feature area, with its required auth level.

All 60+ actions are dispatched through the single `route(action, p)` function in `Code.gs`. An unrecognised action name returns `err('Unknown action')`. The table below is grouped by feature area; the “Auth required” column is the exact role passed to `requireAuth()`, or `None` where no session check exists at all — see the callout in Section 3 for why that matters.

Authentication & Setup

ACTION	AUTH REQUIRED	SHEETS TOUCHED
ping	NO AUTH	—
authenticateUser	NO AUTH	Members
validateSession	NO AUTH	Members
changePassword	NO AUTH	Members
requestPasswordReset	NO AUTH	Members, Gmail
confirmPasswordReset	NO AUTH	Members
initialSetup	NO AUTH	All sheets

Members

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getMembers	MEMBER	Members
getMember	MEMBER	Members
createMember	ADMIN	Members, Notifications, Gmail
updateMember	MEMBER	Members (+R2 if photo)
deleteMember	ADMIN	Members

Assets & Maintenance

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getAssets	NO AUTH	Assets
getAsset	NO AUTH	Assets
createAsset	ADMIN	Assets (+R2)
updateAsset	ADMIN	Assets (+R2)
deleteAsset	ADMIN	Assets

ACTION	AUTH REQUIRED	SHEETS TOUCHED
checkoutAsset	MEMBER	Assets
checkinAsset	MEMBER	Assets
getMaintenanceLogs	MEMBER	MaintenanceLog
addMaintenanceLog	MEMBER	MaintenanceLog, Assets, Notifications, Gmail, FCM
updateMaintenanceLog	MEMBER / ADMIN to complete	MaintenanceLog, Notifications, Gmail, FCM, R2

Notices

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getAnnouncements	NO AUTH	Announcements, Members
createAnnouncement	ADMIN	Announcements, Notifications, Gmail blast, FCM
updateAnnouncement	ADMIN	Announcements
deleteAnnouncement	ADMIN	Announcements
markAnnouncementsRead	MEMBER	Announcements, Members
acknowledgeAnnouncement	MEMBER	Announcements, Members
rsvpAnnouncement	MEMBER	Announcements, Members

Prayer

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getPrayerRequests	NO AUTH	PrayerRequests, Members
createPrayerRequest	MEMBER	PrayerRequests
updatePrayerRequest	MEMBER	PrayerRequests
markAnswered	MEMBER	PrayerRequests, AnsweredPrayers
deletePrayerRequest	MEMBER	PrayerRequests
prayForRequest	MEMBER	PrayerRequests, Notifications, FCM
getAnsweredPrayers	NO AUTH	AnsweredPrayers, Members
getYearEndSummary	NO AUTH	AnsweredPrayers, Members

Birthdays

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getBirthdayWalls	MEMBER	Members, BirthdayWishes
getBirthdayWishes	MEMBER	BirthdayWishes
postBirthdayWish	MEMBER	BirthdayWishes

ACTION	AUTH REQUIRED	SHEETS TOUCHED
sendBirthdayPDFs	SUPER_ADMIN (skipped for trigger)	Members, BirthdayWishes, Settings, Drive, Gmail

Events & Attendance

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getEvents	NO AUTH	Events
createEvent	ADMIN	Events
getAttendance	MEMBER	Attendance
markAttendance	MEMBER	Attendance
markAttendanceBulk	ADMIN	Attendance, Events, Members, Settings, Gmail
getMemberAttendance	NO AUTH	Attendance, Events
exportAttendanceCSV	ADMIN	Events, Members, Attendance

Roster & Prayer Partners

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getPrayerPartners	MEMBER	PrayerPartners, Members
setPrayerPartners	ADMIN	PrayerPartners, Notifications, FCM
autoPairMembers	ADMIN	Members (read)
getFacilitatorRoster	MEMBER	FacilitatorRoster
updateRosterSlot	ADMIN	FacilitatorRoster, Members, Settings, Notifications, Gmail, FCM

Auditions

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getAuditionSuggestions	MEMBER	AuditionSuggestions
createSuggestion	MEMBER	AuditionSuggestions, Members, Notifications, Gmail to admins, FCM
updateSuggestionStatus	ADMIN	AuditionSuggestions, Notifications, FCM

Notifications & Badges

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getNotifications	MEMBER	Notifications, Settings
markNotificationsRead	MEMBER	Notifications
getBadges	MEMBER	Badges
awardBadge	ADMIN (skipped internally)	Badges, Notifications, FCM

Vision, Mission & Onboarding

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getVMContent	NO AUTH	Settings
updateVMContent	SUPER ADMIN	Settings
getQuizQuestions	NO AUTH	Settings
addQuizQuestion	SUPER ADMIN	Settings
updateQuizQuestion	SUPER ADMIN	Settings
saveAllQuizQuestions	SUPER ADMIN	Settings
deleteQuizQuestion	SUPER ADMIN	Settings
recordVMReview	MEMBER	VMReviews
getVMReviewStatus	MEMBER	VMReviews
getOnboardingProgress	MEMBER	OnboardingProgress
updateOnboardingProgress	MEMBER	OnboardingProgress
getOnboardingChecklist	NO AUTH	Settings
addOnboardingItem	SUPER ADMIN	Settings
updateOnboardingItem	SUPER ADMIN	Settings
deleteOnboardingItem	SUPER ADMIN	Settings
reorderOnboardingItems	SUPER ADMIN	Settings
completeOnboarding	MEMBER	Members, VMReviews, Badges, Notifications, Gmail to admins, FCM

System & Settings

ACTION	AUTH REQUIRED	SHEETS TOUCHED
getSettings	NO AUTH	Settings
getAllSettings	SUPER ADMIN	Settings
updateSettings	SUPER ADMIN	Settings
getDashboard	MEMBER	Members, VMReviews, PrayerPartners, Announcements, Badges, Attendance, Events (+admin extras)
exportData	SUPER ADMIN	All sheets (read) + Settings
importData	SUPER ADMIN	All sheets (overwrite)
clearAllData	SUPER ADMIN	All sheets (delete)



SECTION 07

Notifications, Email & Automation

Email templates, push notifications, the five scheduled triggers, and how routing works today versus what's planned.

Every notification — in-app, email, or push — originates from the Apps Script backend. There is no separate notification service; `addNotification()` writes to the `Notifications` sheet and attempts an FCM push in the same call.

Push notifications (Firebase Cloud Messaging)

The client registers a service worker and requests a browser notification permission on login (and again on every fresh session). The resulting FCM device token is saved to the member's `Members` row. Server-side, `getFirebaseAccessToken()` mints a signed JWT from the Firebase service-account key stored in Script Properties, exchanges it for an OAuth2 bearer token, and `sendPushNotification()` posts to the FCM v1 `messages:send` endpoint once per stored device token.

Transactional email templates

All sent via `GmailApp` from the Apps Script project's own Google account — there is no third-party transactional email provider wired into the live backend.

TEMPLATE	SENT WHEN
Welcome	New member account created
Password reset	Reset link with token
Maintenance raised	New maintenance log, sent to submitter and all admins
Maintenance complete	Log marked done
Maintenance reminder (digest)	Multi-row table of currently open logs, used by the 5-hour trigger
Absence	"We missed you at <event>", sent to purely-absent members after bulk attendance is saved
Roster assignment	Facilitator notified of their assigned week, with Zoom details
Announcement blast	Sent to every active member with an email on file when a notice is created
Onboarding complete	Sent to admins when a member finishes onboarding
Audition suggestion	Sent to admins when a member submits a talent suggestion
Vision & Mission	Used around V&M content flows
Birthday PDF	The blessing-wall PDF attachment, see trigger table above

Scheduled triggers

FUNCTION	SCHEDULE	WHAT IT DOES
sendMorningBirthdayNotifications()	Daily, 8–9am IST	In-app notification to every active member listing today's birthdays. No email.
sendBirthdayPDFs()	Daily, after the wish window closes	For each birthday member with an email: builds a Google Doc of their wall's blessings, exports it as a PDF via Drive, emails it as an attachment, then deletes the temporary Doc. Also callable manually by SUPER_ADMIN.
sendMaintenanceReminders()	Every 5 hours	Finds all open (incomplete) maintenance logs and sends a notification + email to every admin and super-admin.
sendMondayPrayerReminder()	Monday morning	Looks up today's roster facilitator and Zoom details, then notifies and emails every active member about that evening's 9:30pm prayer meeting.
sendFridayFacilitatorReminder()	Friday	Finds next Monday's assigned facilitator and sends them a notification + email reminder to prepare, with Zoom details.

Current notification routing model

Today, department-style events fan out broadly rather than to a single owner: a new maintenance log notifies every admin and super-admin (not just an “assets” specialist), and the 5-hour reminder trigger does the same for every open log. There is no escalation logic and no distinction between a routine notification and an overdue one beyond the reminder's own cadence.

PLANNED CHANGE PLANNED

The current engineering brief calls for replacing this all-admin fan-out with routing to the single relevant department admin, a weekly cross-department digest for the Overseer role instead of routine per-item notifications, and a new configurable 48-hour SLA escalation (email + in-app) to the Overseer when a department-actionable item sits unactioned past the threshold. Broadcast-to-everyone triggers (the general prayer reminder, birthday notifications) are explicitly intended to stay as-is. None of this has been built yet — the behaviour described above (“Current notification routing model”) is what a tester should expect to see in the live app today.



SECTION 08

External Integrations

The five external services the backend orchestrates, and how to test each one's failure modes.

The backend orchestrates five external services. The frontend never talks to any of them directly — every integration is mediated by Apps Script, which is both a simplicity win (one set of credentials, one place to change providers) and a single point of failure worth testing deliberately.

Google Apps Script

The entire backend. Deployed as a Web App (“Execute as me, accessible to Anyone”); `doGet`/`doPost` are the only two entry points.

TEST THIS BY

Deployment must be re-published as a **New version** after any code change — saving the script alone does not update the live endpoint. Test failure mode: an un-redeployed change silently continues serving old logic.

Google Sheets

The application database. All reads/writes go through `SpreadsheetApp`, addressed by sheet name and column headers.

TEST THIS BY

No schema migrations exist beyond a handful of idempotent “add missing column” guard functions run on specific actions. Manually renaming/reordering a header will break the corresponding feature silently.

Gmail (GmailApp)

Sends every transactional and blast email listed in Section 7, from the Apps Script project owner's Gmail account.

TEST THIS BY

Subject to Gmail's daily send quota for the account type in use — a large announcement blast plus reminder triggers on the same day could approach it in a large deployment.

Google Drive + Docs

Used only for the daily birthday-PDF flow: a temporary Google Doc is created, exported to PDF, emailed, then the Doc is deleted. Also retains a deprecated, unused photo-storage code path.

TEST THIS BY

If PDF export or Drive cleanup fails partway through, verify no orphaned Docs accumulate in the account's Drive over time.

Cloudflare R2

Primary storage for every profile, asset, and maintenance photo. Uploaded from Apps Script via a hand-rolled AWS Signature Version 4 **PUT** request, using five credentials stored in Script Properties.

TEST THIS BY

SigV4 signing is implemented by hand in Code.gs (date formatting, hex digest, HMAC signing-key derivation) rather than via an SDK — a prime candidate for a subtle bug if credentials or the region ever change.

Firebase Cloud Messaging

Web/device push notifications via the FCM v1 API, using a service-account JWT exchanged for an OAuth2 token on every send.

TEST THIS BY

Push silently no-ops for any member without a valid device token (declined permission, uninstalled PWA, expired token) — the in-app Notifications Centre and email are the fallbacks to verify in that case.



SECTION 09

Non-Functional Requirements

Performance, caching, PWA behaviour, responsiveness, security posture, and image handling.

Performance & caching

- Read actions are cached in `sessionStorage`; mutations call `invalidateCache()` for the affected action before reloading data.
- The `touchCache(sheetName)` pattern lets the frontend cheaply detect “has this sheet changed since I last read it” without re-reading the sheet itself.
- Dashboard data is additionally cached for 60 seconds per member in `sessionStorage`, with pull-to-refresh as the manual override.
- Every authenticated page except Login and Setup is route-split via `React.lazy` to keep the initial load small on mobile networks.

Progressive Web App behaviour

- Installable on Android/desktop via the standard `beforeinstallprompt` flow, with a custom in-app install banner.
- iOS Safari has no native install prompt, so a custom step-by-step install modal appears automatically after 20 seconds on iOS if not previously dismissed.
- A service worker (`vite-plugin-pwa` + a hand-maintained `firebase-messaging-sw.js`) caches the app shell for offline load and handles background push.

Responsiveness

- Mobile: a fixed 5-item bottom nav (Home, Notices, Alerts, Prayer, Roster) plus a floating action button that expands to Assets, Events, Birthdays, Auditions, and (admin only) Settings. Both hide on scroll-down and reappear on scroll-up.
- Desktop: a collapsible left sidebar exposes every route directly, with no FAB.
- Regular members and admins share an identical navigation structure — the only role-gated nav entry is the Settings shortcut.

Theming

A dark/light toggle in the header persists to `localStorage` and applies a `dark` class at the document root; the product's default and primary-tested theme is dark ("midnight") with a gold accent.

Security posture

- Passwords are SHA-256 hashed client-side before transmission and storage — there is no per-user salt, which is worth flagging to anyone extending authentication (see Section 11).
- Session tokens are opaque UUIDs with a server-checked 7-day expiry, validated on every authenticated request via `requireAuth()`.

- Several read endpoints have no auth check at all (Section 3); treat the Apps Script URL itself as sensitive until that gap is closed.
 - The production build pipeline includes an optional obfuscation pass (`npm run build:secure`, via `javascript-obfuscator`) to make the shipped bundle harder to read — this is obscurity, not a substitute for server-side authorisation.
-

Image handling

All photo uploads (profile, asset, maintenance) are compressed client-side before upload: `compressImage()` draws the source image to a canvas capped at 800px on the longest side and iterates JPEG quality downward until the encoded size is under 200KB. This keeps both the R2 storage footprint and the JSON POST payload small on typically slow mobile uploads.



SECTION 10

Test Plan & QA Checklist

A role-based access matrix and 38 concrete, runnable functional test cases drawn directly from the code.

This checklist is written to be run directly — each row is one testable scenario with concrete steps and an expected result, drawn from the business rules documented in Section 4. Run the full role-based matrix first, then work through the functional cases grouped by module.

Test environment

- Point the frontend's Apps Script URL at a **copy** of the production spreadsheet, never the live one — there is no staging backend separate from the Sheet itself.
- Seed at least one account per role (MEMBER, ADMIN, SUPER_ADMIN) and one not-yet-onboarded MEMBER, so role-gated and onboarding-gated behaviour can be exercised without repeatedly editing the sheet by hand.
- Test on at least one Android/desktop Chromium browser (native install prompt) and one iOS Safari (custom install modal) to cover both PWA install paths.

Role-based access matrix — quick pass

Cross-check against the full permission matrix in Section 3. For each row, confirm both that the UI hides the control *and* that the backend action itself rejects the call if attempted directly.

CHECK	MEMBER	ADMIN	SUPER_ADMIN
Can post/edit/delete a notice	No	Yes	Yes
Can add/edit an asset	No	Yes	Yes
Can create an event / mark bulk attendance	No	Yes	Yes
Can assign a roster slot	No	Yes	Yes
Can export attendance CSV	No	No	Yes
Can open the Settings page	No	No	Yes
Can export/import/clear all data	No	No	Yes

Functional test cases

MODULE	ID	SCENARIO	STEPS	EXPECTED RESULT
Login	TC-01	Correct credentials sign in	Enter a valid active member's email + password	Session created, redirected to Dashboard (or Onboarding if not yet onboarded)
Login	TC-02	Wrong password rejected	Enter a valid email with an incorrect password	Generic error shown; no indication of whether the email exists

MODULE	ID	SCENARIO	STEPS	EXPECTED RESULT
Login	TC-03	Inactive account blocked	Attempt login with a deactivated member's credentials	Login fails with the same generic error
Login	TC-04	Password reset round trip	Request reset → open emailed link → set new password (≥8 chars) → log in with it	Old password no longer works; new password does
Onboarding	TC-05	New member forced into onboarding	Log in as a member with isOnboarded=false	Redirected to /onboarding; cannot navigate to any other route until complete
Onboarding	TC-06	Confirmation phrase gate	Type an incorrect phrase on the confirm step	Continue button stays disabled; correct phrase (case/whitespace-insensitive) enables it
Onboarding	TC-07	Checklist completion triggers finish	Check every required checklist item	completeOnboarding fires automatically; confetti shown; member routed into the app with isOnboarded=true
Dashboard	TC-08	Admin-only cards hidden from members	Load Dashboard as MEMBER vs ADMIN	Stat cards, maintenance alerts, V&M tracker, roster/audition summaries appear for ADMIN/SUPER_ADMIN only
Dashboard	TC-09	60-second cache	Reload the Dashboard twice within 60 seconds	Second load renders instantly from cache with no network call; pull-to-refresh forces a fresh one
Notices	TC-10	RSVP No requires a reason	On a Meeting notice, tap No without typing a reason	Confirm button stays disabled until at least 10 characters are entered
Notices	TC-11	Urgency drives response controls	View one notice of each urgency	Info/Urgent show Acknowledge; Meeting shows Yes/No RSVP; Important shows neither
Notices	TC-12	Optimistic update rollback	Acknowledge a notice with the network disabled	UI reverts the optimistic change and shows an error toast
Notices	TC-13	Deep link highlight	Open a link to /notices?id=<id>	Page scrolls to and highlights that notice for ~1.5s
Prayer	TC-14	Anonymous hides identity	Post an anonymous request, view as another member and as an admin	Poster name is hidden everywhere, including to admins
Prayer	TC-15	Pray button debounce	Rapidly tap "Pray for this" multiple times	Only counted once; button disables after the first successful tap
Prayer	TC-16	Testimony minimum length	Attempt to mark a request answered with a 10-character testimony	Submit stays disabled until 20 characters are reached
Prayer	TC-17	Answered archives correctly	Mark a request answered	Moves from Believing to Answered tab; appears in AnsweredPrayers / Year-End Summary
Assets	TC-18	Asset ID generation	Add two assets in the same category	IDs share the category's 3-letter prefix with an incrementing zero-padded number

MODULE	ID	SCENARIO	STEPS	EXPECTED RESULT
Assets	TC - 19	Category-scoped maintenance types	Log maintenance on a Guitar vs a Microphone	Dropdown options differ per category; unmatched categories fall back to the generic list
Assets	TC - 20	Due-date badges	Set nextDueDate to yesterday / in 3 days / in 20 days	Badges read Overdue / Due this week / Due this month respectively
Assets	TC - 21	Photo upload failure recovery	Simulate an R2 upload failure after saving a new asset	Asset record is still saved; user sees a warning toast, not a lost record
Assets	TC - 22	Mark-complete is admin-only	View an open maintenance log as MEMBER vs ADMIN	MEMBER sees a read-only 'Open' badge; ADMIN sees a 'Mark Done' button
Events	TC - 23	Bulk attendance writes all active members	Mark attendance for one event, tapping only a subset of members	Attendance rows exist for every active member, not only the ones tapped, using default Absent for the rest
Events	TC - 24	Absence email targeting	Save attendance with a mix of Present / Informed Prior / Absent	Only purely-Absent members receive the 'We missed you' email; Informed Prior members do not
Events	TC - 25	Re-marking updates, not duplicates	Mark an event's attendance twice	Existing Attendance rows are updated in place, no duplicate rows
Roster	TC - 26	On-break month overrides display	View a roster week that falls in a configured break month	Week shows 'On Break' regardless of any assignment stored for it
Roster	TC - 27	Own-slot highlight	View the roster as a member assigned to an upcoming week	That week shows a 'You!' badge
Birthdays	TC - 28	One wish per person per birthday	Post a blessing, then reload the page	Compose box stays replaced by the 'you left a blessing' confirmation
Birthdays	TC - 29	IST date boundary	Check today's-birthday matching close to midnight IST	Matching uses IST, not the browser's local timezone
Auditions	TC - 30	Member sees only own suggestions	Submit a suggestion as MEMBER A, log in as MEMBER B	MEMBER B does not see MEMBER A's suggestion in their list
Profile	TC - 31	Cannot edit another member	As MEMBER, attempt updateMember with a different member's id	Backend rejects the request; only ADMIN/SUPER_ADMIN can update another member's record
Notifications	TC - 32	Unread badge accuracy	Trigger a new notification, then mark it read	Badge count increments then decrements immediately; count matches the list
Settings	TC - 33	Destructive actions require confirmation	Attempt Clear All Data / Import	Explicit typed confirmation required before the action executes
Cross-cutting	TC - 34	Session expiry enforced server-side	Manually expire a sessionToken's sessionExpiry in the sheet, then call any authenticated action	Request is rejected even though the client still holds the token
Cross-cutting	TC - 35	Unauthenticated read endpoints	Call getAssets / getAnnouncements / getPrayerRequests / getEvents directly with no session token	Data is returned with no auth error — confirm this matches the documented gap in Section 3, not a regression

MODULE	ID	SCENARIO	STEPS	EXPECTED RESULT
Cross-cutting	TC-36	Push permission declined	Deny the browser notification permission on login	App continues to function normally; notifications still appear in the in-app Notifications Centre
Cross-cutting	TC-37	Apps Script redeploy discipline	Change a handler in Code.gs, save only (no new deployment)	Live endpoint continues serving the old behaviour until a New Version is deployed
Cross-cutting	TC-38	Offline app shell	Load the app once online, then reload with the network disabled	Installed PWA shell still renders from the service worker cache rather than a browser offline error

Regression checklist for every release

- Mobile bottom nav still shows exactly five items with Alerts centred, and the FAB still opens to Assets, Events, Birthdays, Auditions (+ Settings for admins) — this is an explicit non-negotiable in the current engineering brief and should never change incidentally.
- A brand-new MEMBER account's first-run experience is unchanged: straight into onboarding, no extra nav items, no new permission prompts.
- Cache invalidation: after any mutation, confirm the affected list re-renders with fresh data rather than a stale cached copy.
- Toast/error handling: force at least one network failure per module and confirm a toast appears rather than a silent failure or an unhandled exception.



SECTION 11

Known Gaps & Technical Debt

Documented current-state limitations and trade-offs — read before filing a new defect.

These are not bugs in the sense of “the app is doing something it shouldn’t” — each item below is either a deliberate trade-off, a documented current-state limitation, or planned work not yet started. They are collected here so a developer picking up this codebase does not rediscover them the hard way, and so a tester does not mistakenly file them as new defects.

UNAUTHENTICATED READ ENDPOINTS

`getAssets`, `getAsset`, `getAnnouncements`, `getPrayerRequests`, `getAnsweredPrayers`, `getYearEndSummary`, `getEvents`, and `getMemberAttendance` have no `requireAuth()` call and will return data to anyone with the Apps Script URL. Documented in the codebase's own architecture notes as a live, current-state gap.

PASSWORD HASHING HAS NO PER-USER SALT

SHA-256 is applied to the raw password with no salt, client-side. This is materially weaker than a proper salted/adaptive hash (e.g. `bcrypt`/`scrypt`/`Argon2`) and would be worth revisiting if the member list or credential set ever needed to withstand a serious offline attack.

DEPRECATED DRIVE PHOTO PATH RETAINED

`saveImageToDrive()` still exists in `Code.gs` for rollback purposes but is superseded by `uploadToR2()` for all current photo uploads. Dead code, low risk, but worth removing once R2 is fully trusted in production.

HAND-ROLLED AWS SIGV4 SIGNING

Cloudflare R2 uploads are signed by hand in Apps Script (date formatting, hex digest, HMAC key derivation) rather than via a vetted SDK. Any change to R2 credentials, bucket region, or endpoint format is a plausible source of a hard-to-diagnose upload failure.

ATTENDANCE-MARKING REWRITES ALL ACTIVE MEMBERS

`markAttendanceBulk()` writes/updates a row for every active member on every save, not just the ones the admin interacted with. Functionally correct today, but means a member added after an event was marked will not get a retroactive row until the event is re-marked.

DEAD/DISABLED DASHBOARD FEATURES

Birthdays-on-dashboard, Badges display, and the personal attendance bar are implemented in `Dashboard.jsx` but explicitly disabled behind `{false && ...}` guards. They should not be mistaken for missing functionality during testing, nor silently re-enabled without a product decision.

DOCUMENTATION DRIFT: EMAILJS VS GMAILAPP

The project's own change-list document (docs/CHANGE_LIST.md) references EmailJS as the email mechanism in one example. The actual, current backend sends all transactional and blast email via GmailApp; there is no EmailJS integration in Code.gs. This document follows the code.

DEPARTMENT-ADMIN ROLE MODEL NOT YET BUILT

Section 3 documents the planned MAIN_ADMIN / department-admin model in detail. As of this document's date, the Members sheet still stores a single flat role enum and no department scoping, escalation, or digest logic exists in Code.gs.

BUILD-TIME OBFUSCATION IS NOT A SECURITY BOUNDARY

npm run build:secure obfuscates the shipped JS bundle to raise the bar for casual inspection, but all real authorisation must continue to happen server-side in requireAuth() — obfuscation does not substitute for it.



SECTION 12

Deployment & Release Process

Build and deploy steps for both the frontend and the backend, and the one step most releases miss.

There are two independent deployables — the frontend (Cloudflare Pages) and the backend (Apps Script Web App) — and neither redeploys the other automatically.

Frontend build & deploy

COMMAND	WHAT IT DOES
<code>npm run dev</code>	Local development server (Vite)
<code>npm run build</code>	Production build to <code>dist/</code>
<code>npm run build:secure</code>	Production build, then runs <code>build-tools/obfuscate.js</code> (javascript-obfuscator) over the output
<code>npm run deploy</code>	Alias for <code>build:secure</code> — the output is then published to Cloudflare Pages
<code>npm run preview</code>	Serves the built <code>dist/</code> locally for a final check before publishing

Cloudflare Pages serves the static `dist/` output directly, with `_redirects` handling single-page-app routing so client-side routes resolve correctly on a hard refresh.

Backend deploy

THE STEP MOST LIKELY TO BE MISSED

Saving changes in the Apps Script editor does **not** update the live Web App endpoint. A **New deployment version** must be published explicitly (Deploy → Manage deployments → Edit → New version) for any backend change to take effect in production. Every task in the current engineering brief calls this out as a required manual step after each change — treat it as part of the definition of done for any backend PR.

Required environment configuration

VARIABLE	PURPOSE
<code>VITE_APPS_SCRIPT_URL</code>	The deployed Web App URL (can also be set from within the app via the Setup Wizard)
<code>VITE_FIREBASE_API_KEY</code> and five related <code>VITE_FIREBASE_*</code> vars	Firebase web app config, from Firebase Console → Project Settings
<code>VITE_FIREBASE_VAPID_KEY</code>	Web Push certificate key pair, from Firebase Console → Cloud Messaging

Backend-side credentials (R2 access keys, Firebase service-account key) live in Apps Script Script Properties, not in the frontend `.env`.

Reference setup guides in the repository

[docs/APPS_SCRIPT_SETUP.md](#) · [docs/CLOUDFLARE_DEPLOY.md](#) · [docs/FIREBASE_SETUP.md](#) · [docs/MEMBER_ONBOARDING_GUIDE.md](#) · [docs/OBFUSCATION_GUIDE.md](#)



SECTION 13

Appendix

Glossary, roadmap summary, and document revision history.

Glossary

V&M

Vision & Mission — the team's foundational content every member reads and signs off on at onboarding and monthly thereafter.

IST

Indian Standard Time (UTC+5:30). Birthday matching, roster weeks, and the “now” timestamp helper all compute against IST regardless of the visiting device's local timezone.

FCM

Firebase Cloud Messaging — Google's push-notification service, used here via its v1 HTTP API.

R2

Cloudflare R2 — S3-compatible object storage; holds every profile, asset, and maintenance photo.

SigV4

AWS Signature Version 4 — the request-signing scheme R2's S3-compatible API requires, implemented by hand in Code.gs.

Apps Script Web App

A Google Apps Script project deployed as an HTTP endpoint via `doGet/doPost`; the entirety of this app's backend.

Overseer

The planned MAIN_ADMIN role — a cross-department oversight role that receives a weekly digest and 48-hour escalation alerts rather than routine per-item notifications.

touchCache pattern

A lightweight change-detection mechanism: mutating handlers write a `cacheTs_<Sheet>` timestamp to Settings so the frontend can decide whether its sessionStorage cache is stale.

Roadmap summary PLANNED

The current engineering brief defines two sequential tasks, with Task 1 as a prerequisite for Task 2. Both are described in full in Section 3 and Section 7; summarised here for reference.

TASK	SUMMARY
Task 1 — Department-scoped admin roles	Introduce MAIN_ADMIN (Overseer) and four department-admin grants (Worship / Assets / Auditions / Prayer) alongside the existing MEMBER/ADMIN/SUPER_ADMIN enum, migrate existing admins to the closest department(s), and close any remaining server-side role-enforcement gaps.
Task 2 — Notification routing overhaul	Route department-actionable notifications to the single relevant department admin instead of all admins; give the Overseer a weekly digest plus 48-hour SLA escalation; consolidate recurring per-item admin reminders into single digest notifications.

Both tasks are explicitly scoped as additive to the admin/backend side only — the base member experience, including the mobile FAB menu, is a hard constraint that must not change as a side effect of either task.

Document revision history

VERSION	DATE	NOTES
1.0	25 July 2026	Initial version, produced from a direct review of the live codebase